

Reviewer Pack — Minimal Quadratic Entanglement in Quantum States and Communi...

Entry 1656dabd2aa6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 08:52:15 UTC

Minimal Quadratic Entanglement in Quantum States and Communication Complexity

Entry ID: 1656dabd2aa6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 08:52:15 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Quantum Entanglement) **Field B** (complexity object): Communication Complexity

Statement:

For any given quantum state ρ representing a bipartite system with dimensions d_1 and d_2 , the minimal entanglement entropy between the two parts satisfies $|S(\rho)| = \Omega(\log(\min(d_1, d_2)))$ where $S(\rho)$ is the von Neumann entropy of ρ .

Rationale (proposer's reasoning):

Quantum entanglement measures the degree of correlation in a quantum system. It has been shown that entanglement is a resource for communication complexity tasks. This conjecture links the quantum entanglement entropy directly to communication complexity, suggesting that the minimal entanglement required for efficient communication grows logarithmically with the minimum dimension of the bipartite system.

Taxonomy category: Quantum_Information_Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dbe2064a3cfa463c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given bipartite system with dimensions d_1 and d_2 , if the von Neumann entropy of the quantum state ρ satisfies $|S(\rho)| \geq 0.5 * \log(\min(d_1, d_2))$ for all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal quadratic entanglement" AND "communication complexity" AND quantum information theory - "von Neumann entropy" AND bipartite system AND communication complexity IN quantum information theory - "entanglement entropy" AND $\log(\min(d_1, d_2))$ AND quantum entanglement AND communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_unitary(d):
        U = [[random.gauss(0, 1) for _ in range(d)] for _ in range(d)]
        Q, R = qr_decomposition(U)
        return Q

    def qr_decomposition(A):
        m, n = len(A), len(A[0])
        Q = [[A[i][j] for j in range(n)] for i in range(m)]
        R = [[0 if i != j else 1 for j in range(n)] for i in range(m)]

        for k in range(min(m, n)):
            norm = sum(Q[k][i]**2 for i in range(k, m))**0.5
            Q[k] = [Q[k][i] / norm for i in range(m)]

            for j in range(k+1, n):
                R[k][j] = sum(Q[k][i] * Q[i][j] for i in range(k, m))
                Q[j] = [Q[j][i] - R[k][j] * Q[k][i] for i in range(m)]

        return Q, R

    def von_neumann_entropy(rho):
        eigenvalues = [rho[i][i] for i in range(len(rho))]
        entropy = sum(-eigenvalue * math.log2(eigenvalue) if eigenvalue > 0 else 0 for eigenvalue
            in eigenvalues)
        return entropy

    n_values = [5, 10, 15, 20, 30, 40]
```

```

results = []

for n in n_values:
    d1 = random.randint(2, n)
    d2 = random.randint(2, n)

    U = generate_unitary(d1)
    V = generate_unitary(d2)
    rho = [[U[i][k] * V[j][l] for l in range(d2)] for k in range(d1)]

    entropy = von_neumann_entropy(rho)
    min_dim = min(d1, d2)
    results.append((n, entropy, math.log2(min_dim)))

if not results:
    return {
        "metric_name": "log_min_dimension",
        "metric_value": 0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

log_min_dims = [r[2] for r in results]
entropies = [r[1] for r in results]

mean_entropy = sum(entropies) / len(entropies)
std_entropy = (sum((e - mean_entropy)**2 for e in entropies) / len(entropies))**0.5
support_fraction = sum(e >= 0.5 * d for n, e, d in results) / len(results)

return {
    "metric_name": "log_min_dimension",
    "metric_value": mean_entropy,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": "" if support_fraction >= 0.8 else "support_fraction < 0.8"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 7 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))**0.5
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")

```

```

elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='support_fraction < 0.8' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f3b0b66.py", line 93, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f3b0b66.py", line 55, in run_trial
    rho = [[U[i][k] * V[j][l] for l in range(d2)] for k in range(d1)]
            ^
NameError: name 'i' is not defined. Did you mean: 'id'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to an undefined variable 'i', which means that the pre-registered support condition was not tested and thus cannot be unambiguously confirmed. | next: Review the test code for errors, particularly the definition of the variables used in the calculation of the von Neumann entropy and correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18745
2	propose	ollama_remote	glm4:latest	0	0	20607
3	preregistration	ollama_remote	glm4:latest	0	0	13713
4	novelty	ollama_remote	glm4:latest	0	0	8449
5	novelty	ollama_remote	glm4:latest	0	0	9547
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15653
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10550
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11426
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12160
10	judge	ollama_remote	glm4:latest	0	0	13671

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 134521 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/1656dabd2aa6.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/1656dabd2aa6.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1656dabd2aa6.tar.gz (if generated)